SIMATS ENGINEERING
ASSESSMENT TOOL 2

Name: Navuluri Anjali
Reg No: 192324137
Marks: 94/100

COURSE NAME: INTERNET PROGRAMMING **COURSE CODE: CSA43**

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I Navuluri Anjali (192324137) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Anjali*

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	24
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	24
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	19
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	18
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	9

Question 1: HTTP Response Analysis

Given HTTP Response Header
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

1. Identify Any Incorrect or Invalid Fields in the Header

The given HTTP response header contains two errors: an incorrect weekday in the Date field and an invalid value in the Content-Length field.

- **HTTP/1.1 200 OK:** Indicates that the request was successfully processed.
- **Date:** April 1, 2026, falls on a Wednesday, not Monday. Therefore, the weekday is incorrect.
- **Content-Type: text/html:** Correctly specifies that the response contains HTML content.
- **Content-Length: -150:** Invalid because the content length cannot be negative.
- **Connection: keep-alive:** Indicates that the connection may remain open for subsequent requests.

Thus, the Date and Content-Length fields must be corrected.

2. Analyze the Issue with Content-Length and Explain Its Impact

The Content-Length header specifies the size of the HTTP response body in bytes. In the given response, the value is -150, which is invalid because the length of a response body cannot be negative. This error may cause the browser or other HTTP clients to reject the response or handle it incorrectly. It can also create problems when interpreting the response body and its boundaries.

Incorrect code:
Content-Length: -150
Corrected code (assuming the body is 150 bytes):
Content-Length: 150
The value must match the actual size of the response body in bytes.

3. Determine Whether the Status Code Matches the Response Correctness

The status code 200 OK indicates that the server has successfully processed the request and is returning a successful response.
However, a successful status code does not guarantee that all response headers are valid. In this case, the invalid Content-Length value makes the response header malformed.
The server can retain 200 OK if the request was successful and the response is otherwise valid. The header errors must be corrected separately.
Therefore, the status code is appropriate for a successful request, but the response header requires correction.

4. Suggest Corrections for the Identified Errors

The server should correct the weekday in the date field and replace the negative content length with the actual response body size.

Corrected HTTP response header:
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive

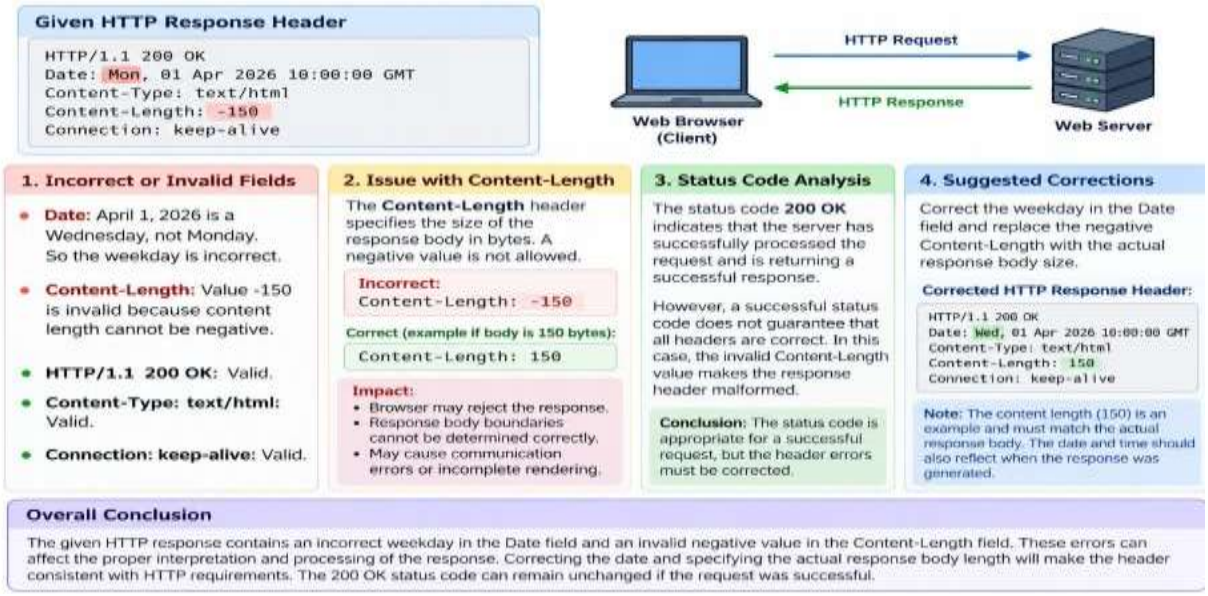


Figure 1: HTTP Response Header Analysis and Error Correction

Note: The content length of 150 is an example and must match the actual response body. The date and time should also reflect when the response was generated.
The server should validate response headers before sending them to the client. Testing tools and browser developer tools can help identify malformed headers and other HTTP communication errors.

Conclusion
The given HTTP response contains an incorrect weekday in the Date field and an invalid negative value in the Content-Length field. These errors can affect the proper interpretation and processing of the response. Correcting the date and specifying the actual response body length will make the header consistent with HTTP requirements. The 200 OK status code can remain unchanged if the request was successful.

QUESTION 2: DOM Structure Analysis

Question:
Consider the following DOM tree representation: CSA43-CO1-MEDIUM - Data Interpr…
Given HTML Code

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Answer the following questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

---

1. Identify Missing or Improperly Closed Elements

The given HTML code contains an unclosed paragraph element ( ).

- ** tag:** The paragraph begins with HelloWorld, but the closing tag is missing.
- ** tag:** The element is properly closed using .
- **<body> and <html> tags:** Both are properly closed.
- **<head> tag:** It is correctly closed, and the <title> element is also properly closed.

Therefore, the main error is the missing closing tag.

2. Analyze How the Browser Will Interpret This Structure

Browsers use the HTML parser to convert HTML code into a Document Object Model (DOM) tree. When the browser encounters an unclosed paragraph element, it attempts to recover from the error and construct a usable DOM tree.
For example, when the browser encounters , it generally closes the open paragraph automatically before closing the element.
The browser may therefore interpret the structure approximately as follows:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld</p>
    </div>
  </body> </html>
```

This automatic error recovery allows the browser to display the page instead of stopping completely.

3. Determine the Impact on Rendering and Layout

The missing closing tag may affect the DOM structure and how the webpage is displayed.

- **Unexpected DOM structure:** The browser automatically repairs malformed HTML, so the resulting DOM may differ from the original source.
- **Layout issues:** In more complex pages, improperly nested elements can affect spacing, alignment, and element positioning.
- **CSS styling problems:** Styles applied to paragraph elements may produce unexpected results if the structure differs from what the developer intended.
- **JavaScript behavior:** JavaScript that accesses or modifies the DOM may behave unexpectedly if the actual DOM structure is different from the expected structure.

In this simple example, the browser will usually display HelloWorld correctly. However, writing valid HTML is important for predictable rendering and easier maintenance.

4. Suggest Corrections to Make the DOM Valid

The missing closing </p> tag should be added before the closing tag.

Corrected HTML Code

```
<!DOCTYPE html>
<html>
<head>
   <title>TestPage</title>
</head>
<body>
   <div>
      <p>HelloWorld</p>
   </div>
</body>
</html>
```

Explanation of Corrections

1. Added <!DOCTYPE html> to declare an HTML5 document.
2. Added </p> to close the paragraph element properly.
3. Maintained the correct order of the , <body>, and <html> closing tags.
4. Ensured that the HTML structure is clear and easy to maintain.

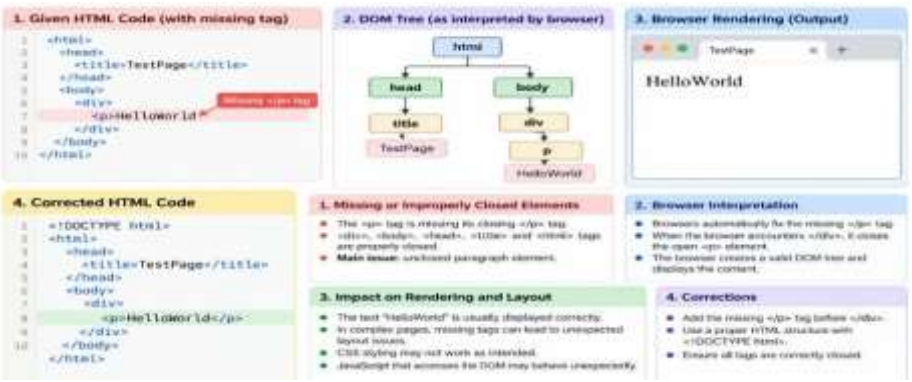
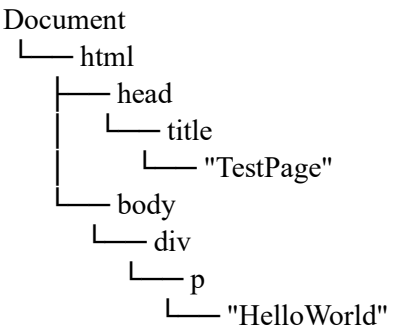


Figure 2: DOM Structure Analysis and HTML Error Correction

Additional Analysis: Importance of a Proper DOM Structure

The Document Object Model (DOM) represents an HTML document as a tree of elements. Each HTML element becomes a node, and the relationship between parent and child elements determines the document structure.
In the given example, the element acts as the parent of the element. Since the paragraph's closing tag is missing, the browser automatically attempts to correct the structure while parsing the HTML.

Expected DOM structure:



This tree shows the expected structure after the browser corrects the missing paragraph closing tag.

Additional Points on Browser Rendering

Browsers are designed to handle common HTML errors through a process called *error recovery*. Therefore, the missing tag does not necessarily prevent the webpage from displaying.
However, developers should not rely on automatic correction because:

- **Maintainability:** Properly structured HTML is easier to read, understand, and modify.
- **CSS compatibility:** Correct nesting helps styles apply to the intended elements.
- **JavaScript functionality:** A predictable DOM structure makes it easier to select and manipulate elements.
- **Cross-browser consistency:** Valid HTML reduces the possibility of differences in how browsers interpret malformed code.
- **Accessibility:** Well-structured HTML helps assistive technologies interpret webpage content more reliably.

Conclusion

The given HTML code contains a missing closing paragraph tag. Browsers can usually recover from this error and display the content, but malformed HTML may cause unexpected DOM structures and rendering issues in more complex applications. Adding the missing tag ensures a well-structured HTML document and improves the reliability of CSS styling and JavaScript operations.

QUESTION 3: GET vs POST in Login System

Question:
A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Sent in the request body	Moderate

Answer the following questions: CSA43-CO1-MEDIUM - Data Interpr… 1.
Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

---

1. Interpret the Differences in Data Transmission Between GET and POST

GET and POST are HTTP methods used by web applications to communicate with servers. However, they differ in how they transmit data and how they are typically used.

GET Method
The GET method sends data through URL query parameters. The data appears in the address bar after the question mark (?).
Example:
GET /login?user=admin&pass=1234 HTTP/1.1
Host: example.com
In this example, the username and password are included in the URL. This makes the credentials more likely to be exposed through browser history, server logs, and copied links.
GET is primarily intended for retrieving resources rather than submitting sensitive information.

POST Method
The POST method sends data in the request body instead of placing it in the URL.
Example:
POST /login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded user=admin&pass=1234

In this example, the username and password are transmitted in the request body. They do not appear directly in the browser's address bar.
POST is generally more suitable for submitting login credentials because it avoids exposing them in the URL. However, **POST alone does not encrypt the data**; HTTPS is required to protect credentials during transmission.

Key Differences

Feature	GET	POST
Data transmission	Through URL parameters	Through the request body

Data visibility	Visible in the URL	Not displayed in the URL
Browser history	Parameters may be stored	Request body is not normally stored in browser history
Data security	Greater risk of credential exposure	Reduces URL-related exposure
Common purpose	Retrieving resources	Submitting data
Login suitability	Not recommended	Recommended with HTTPS

2. Analyze Which Method Is More Suitable for Login and Why

The **POST method is more suitable for login authentication** because it sends credentials in the request body rather than exposing them directly in the URL.

The following reasons explain why POST is preferred:

1. Reduced exposure of credentials:
Username and password values are not displayed in the address bar, reducing the risk of accidental disclosure through copied URLs or browser history.
2. Better handling of sensitive data:
POST is designed for submitting data to a server, making it appropriate for sending login forms.
3. Compatibility with secure authentication:
POST can be combined with HTTPS to encrypt the communication between the browser and server.
4. Server-side validation:
The server can validate the submitted credentials against stored user information and grant access only when authentication succeeds.

Example of a login form using POST:

```
<form action="/login" method="POST">
   <label>Username:</label>
   <input type="text" name="username" required>

   <label>Password:</label>
   <input type="password" name="password" required>

   <button type="submit">Login</button>
</form>
```

In this example, the form submits the username and password to the /login endpoint using POST.
Therefore, POST is preferred for login systems, but it must be combined with HTTPS and proper server-side security measures.

3. Identify Security Risks in Using GET for Authentication

Using GET to transmit login credentials introduces several security risks because the information is included in the URL.

Major Security Risks

1. Exposure through browser history:
URLs containing usernames and passwords may remain in browser history, allowing other people using the same device to discover the credentials.
2. Server log exposure:
Web servers, reverse proxies, and monitoring systems may record request URLs. If credentials are included in the URL, they may be stored in these logs.
3. Accidental sharing:
Users may copy and share a URL containing login credentials without realizing that sensitive information is included.
4. Referrer leakage:
Depending on browser policies and application behavior, URL information may be disclosed through referrer headers when navigating to other resources.
5. Network interception without HTTPS:
If HTTP is used without TLS encryption, attackers monitoring the network may capture the credentials. This risk applies to both GET and POST.
Example of an insecure login URL:
https://example.com/login?user=admin&pass=1234
This URL exposes the password in the query string even though HTTPS is used. HTTPS protects the URL during transmission, but it does not prevent the URL from appearing in browser history or being recorded by systems that process it.
Therefore, GET should not be used to transmit passwords or other sensitive authentication information.

4. Suggest Best Practices for Secure Login Implementation

A secure login system should protect credentials during transmission, validate users correctly, and prevent unauthorized access.

The following practices should be implemented:
1. Use POST for login forms:
Send credentials in the request body instead of including them in URL parameters.
2. Implement HTTPS:
Use HTTPS with valid TLS certificates to encrypt communication between the client and server.
3. Hash passwords securely:
Store passwords using a suitable password-hashing algorithm, such as Argon2id or bcrypt, rather than storing them as plain text.
4. Validate credentials on the server:
Verify the submitted username and password on the server. Client-side validation alone is not sufficient.
5. Use secure session management:
After successful authentication, use properly managed session identifiers or secure tokens. For session cookies, enable attributes such as HttpOnly, Secure, and an appropriate SameSite policy.
6. Prevent brute-force attacks:
Apply rate limiting, login-attempt monitoring, and appropriate protections against repeated passwordguessing attempts.

7. Avoid exposing sensitive information:
Do not include passwords in URLs, application logs, error messages, or analytics records.
8. Implement multi-factor authentication (MFA):
Require an additional verification factor where appropriate to provide extra protection if a password is compromised.

Secure Login Process

The following flow illustrates a basic secure login process:
User enters username and password
|
v
Browser sends POST request over HTTPS
|
v
Server receives login credentials
|
v
Server verifies the credentials
|
+-----+-----+
| |
Valid Invalid
| |
v v
Create secure Reject login
session and show error
|
v
Grant access to protected page
This process helps ensure that credentials are transmitted securely and verified before access is granted.

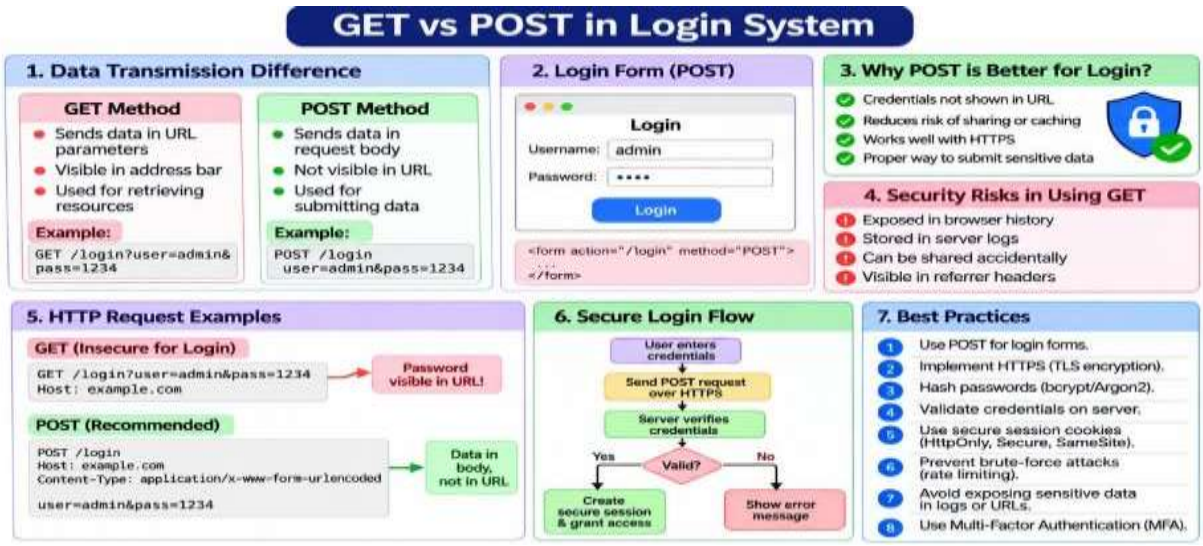


Figure 3: GET vs POST in Login System – Data Transmission, Security Risks, and Secure Login

Implementation

Conclusion

GET and POST are HTTP methods used for different purposes in web applications. GET sends parameters through the URL, whereas POST sends submitted data in the request body. Using GET for authentication is unsafe because credentials may be exposed through browser history, server logs, and shared links.
POST is the preferred method for login forms because it avoids placing credentials directly in the URL. However, POST does not provide encryption by itself. A secure login system must also use HTTPS, secure password hashing, server-side validation, appropriate session management, and protection against repeated login attempts.
Thus, combining POST with HTTPS and proper authentication practices helps protect user credentials and improve the overall security of a login system.

QUESTION 4: HTML Code Inspection

Question:
Analyze the following HTML snippet. CSA43-CO1-MEDIUM - Data Interpr…
Given HTML Code

```
<!DOCTYPE html>
<html>
<head>
   <title>Sample</title>
</head>
<body>
   <h1>Welcome
   <p>This is a paragraph
   <img src="image.jpg">
</body>
</html>
```

Answer the following questions:

1. Identify missing closing tags.
2. Analyze issues with the <img> tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

---

1. Identify Missing Closing Tags

The given HTML code contains missing closing tags for the heading and paragraph elements.

- **<h1> tag:** The heading starts with <h1>Welcome, but the closing </h1> tag is missing.
- ** tag:** The paragraph starts with This is a paragraph, but the closing tag is missing.

- **<img> tag:** The <img> element does not require a closing tag in HTML because it is a void element.
- **Other tags:** The <title>, <head>, <body>, and <html> elements are properly closed. Therefore, the missing closing tags are </h1> and </p>.

2. Analyze Issues with the <img> Tag

The <img> tag is used to display an image on a webpage. In the given code, the image element is:
<img src="image.jpg">
This tag contains the src attribute, which specifies the location of the image file.

However, there are some issues to consider:

1. **Missing alt attribute:**

The image does not have an alternative text description. The alt attribute improves accessibility by describing the image to users who rely on screen readers. It also provides alternative text if the image cannot be displayed.

2. **Image file availability:**

The file image.jpg must exist at the specified location. If the browser cannot find the file, the image will not display correctly.

3. **Image path:**

If the image is stored in another folder, the correct relative or absolute path must be provided.

Corrected image tag:

<img src="image.jpg" alt="Sample image">

The alternative text should describe the actual image. If the image is purely decorative, an empty alt attribute (alt="") is appropriate.

3. Determine How Browsers Will Handle These Errors

Web browsers use an HTML parser to interpret HTML code and construct the Document Object Model (DOM).
When the browser encounters missing closing tags, it attempts to recover from the errors and display the webpage.
The browser will generally:

- Display the heading text Welcome.
- Display the paragraph text This is a paragraph.
- Attempt to load the image from image.jpg.
- Display a broken-image indicator or other fallback if the image cannot be loaded.

Browsers often infer missing closing tags, but relying on this behavior can produce unexpected results in more complex documents. Missing alternative text can also reduce accessibility.
Therefore, developers should use valid HTML to ensure predictable rendering and better accessibility.

4. Suggest a Corrected Version of the Code

The missing closing tags should be added, and the image should include an appropriate alt attribute.

Corrected HTML Code

```
<!DOCTYPE html>
<html>
<head>
   <title>Sample</title>
</head>
<body>
   <h1>Welcome</h1>
   <p>This is a paragraph.</p>
   <img src="image.jpg" alt="Sample image">

</body>

</html>
```

Explanation of Corrections

1. Added </h1> to close the heading element.
2. Added </p> to close the paragraph element.
3. Added the alt attribute to improve image accessibility.
4. Retained the correct HTML document structure.
5. Ensured that all required closing tags are present and properly arranged.

The image file must also be available at the specified path for it to appear on the webpage.

Additional Analysis: Importance of Valid HTML

Valid HTML helps browsers interpret webpage content consistently. Missing tags may be automatically corrected by the browser, but the resulting DOM might not always match the developer's intention. Properly structured HTML offers several benefits:

- **Readability:** Makes the code easier to understand and maintain.
- **Accessibility:** Helps assistive technologies interpret webpage content.
- **Rendering:** Reduces unexpected layout and formatting issues.
- **Debugging:** Makes errors easier to identify and correct.
- **Compatibility:** Encourages consistent behavior across browsers.

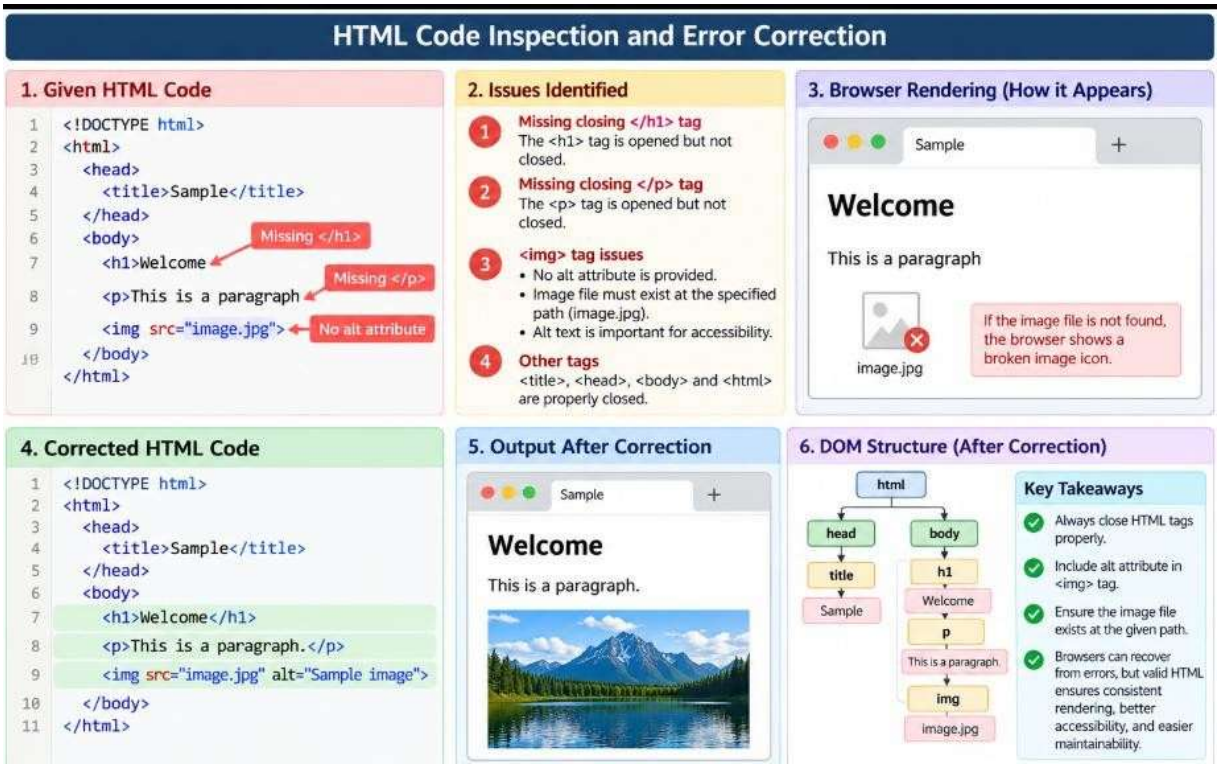


Figure 4: HTML Code Inspection and Error Correction

Conclusion

The given HTML code contains two missing closing tags: </h1> and </p>. The <img> element also lacks an alt attribute, and the image file must be available at the specified path. Although browsers can recover from many HTML errors, these corrections improve document structure, accessibility, and maintainability. Using properly structured HTML ensures that webpages are easier to develop, debug, and display consistently.

QUESTION 5: Browser–Server Communication Flow

Given Sequence:

1. User enters a URL in the browser.
2. DNS lookup occurs.
3. HTTP request is sent.
4. Server processes the request.
5. Response is returned.
6. Browser renders the page.

Answer the following questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

---

1. Interpret Each Step in the Communication Flow

Browser–server communication is the process through which a web browser requests information from a web server and displays the received content to the user. This process involves several steps.

Step 1: User Enters a URL
The process begins when a user enters a website address, such as https://www.example.com, into the browser's address bar.
The browser identifies the required resource and determines the protocol to use. If HTTPS is specified, the browser establishes a secure connection with the server before exchanging HTTP messages.

Step 2: DNS Lookup
The Domain Name System (DNS) translates the domain name into an IP address. For example:

```
www.example.com
      |
v
  DNS Lookup
      |
v
  IP Address
```

The browser uses the IP address to locate the appropriate server. If the DNS result is already cached and valid, the browser may not need to perform a new DNS lookup.

Step 3: HTTP Request Is Sent
After locating the server, the browser establishes the required network connection and sends an HTTP request.
Example:

```
GET /index.html HTTP/1.1
Host: www.example.com
Accept: text/html
```

Here, GET requests the webpage, while the Host header identifies the target website.

Step 4: Server Processes the Request
The web server receives the request, identifies the requested resource, and processes it.
The server may retrieve an HTML file, access a database, execute application logic, or communicate with other services before preparing the response.

Step 5: Response Is Returned
After processing the request, the server sends an HTTP response containing a status code, response headers, and, when applicable, a response body.
Example:
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1200
The 200 OK status indicates success. The response body contains the requested HTML content, assuming the response is a normal HTML page.

Step 6: Browser Renders the Page
The browser processes the received HTML and constructs the Document Object Model (DOM). It also processes CSS, executes JavaScript, and loads additional resources such as images and fonts.
Finally, the browser displays the webpage to the user.
Thus, browser–server communication allows users to access websites through a sequence of name resolution, request transmission, server processing, response delivery, and rendering.

2. Identify Where Delays Can Occur

Delays can occur at different stages of browser–server communication and may affect the overall loading time of a webpage.

Stage	**Possible Cause of Delay**	**Impact**
DNS lookup	Slow DNS server or cache miss	Delays locating the server
Connection establishment	Network latency or packet loss	Delays establishing a connection
TLS handshake	Additional communication required for HTTPS	Delays secure data transfer
Request transmission	Slow network or congestion	Delays delivery of the request
Server processing	Complex operations or slow database queries	Delays response generation
Response transmission	Large files or limited bandwidth	Delays downloading content
Browser rendering	Large images, complex CSS, or heavy JavaScript	Delays displaying the page

Explanation
DNS delay: The browser may take additional time to resolve a domain name into an IP address. **Network delay:** A slow internet connection or network congestion can increase the time required to exchange data.
Server delay: Heavy server workloads, inefficient application code, and slow database queries can increase response time.

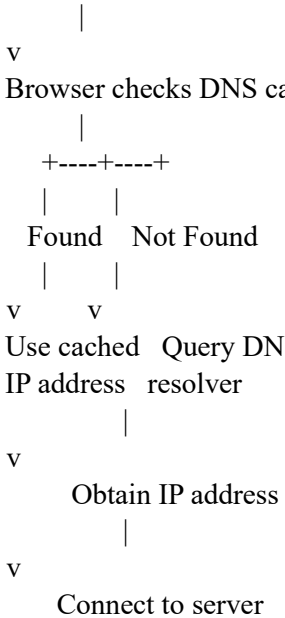
Rendering delay: Large images, excessive JavaScript, and complex page layouts may delay the display of the webpage.
Therefore, identifying the stage responsible for a delay helps developers choose the appropriate performance optimization.

3. Analyze the Role of DNS in the Process
The Domain Name System (DNS) is an essential component of browser–server communication. It translates human-readable domain names into IP addresses that computers use to communicate over a network.
For example, users can enter www.example.com instead of remembering its numerical IP address.

Functions of DNS

1. **Domain name resolution:** Converts a domain name into the corresponding IP address.
2. **Server identification:** Helps the browser locate the server hosting the requested website.
3. **Caching:** Stores DNS results temporarily to reduce repeated lookups and improve performance.
4. **Load distribution:** DNS can return different IP addresses to distribute traffic across multiple servers, depending on the DNS configuration.
5. **Efficient access:** Allows users to access websites using easy-to-remember domain names.

DNS Resolution Process



If a valid cached DNS result exists, the browser can use it directly. Otherwise, it may need to query a DNS resolver to obtain the IP address.

Impact of DNS on Performance
A slow DNS lookup increases the time required to begin connecting to a website. DNS caching reduces repeated lookups and helps improve page-loading speed.
However, DNS does not transfer the webpage itself. Its primary role is to help the browser identify the destination server.
Thus, DNS provides an important link between the domain name entered by the user and the server that delivers the requested webpage.

3. Suggest Optimizations to Improve Performance

Web developers can improve browser–server communication by reducing unnecessary network requests, optimizing server operations, and minimizing the amount of data transferred.

1. Use DNS Caching
DNS caching stores domain-resolution results for a specified period. It reduces repeated DNS queries and helps the browser locate servers faster.
2. Use a Content Delivery Network (CDN)
A CDN distributes cached website resources across servers in different geographical locations. Delivering content from a nearby server can reduce latency.
3. Optimize Server Processing
Efficient application code, database indexing, and optimized database queries help reduce server response time.
4. Compress Response Data
Using suitable compression, such as Gzip or Brotli, reduces the size of text-based resources, including HTML, CSS, and JavaScript.
This reduces the amount of data transferred over the network.
5. Minimize HTTP Requests
Combining resources where appropriate and removing unnecessary files can reduce the number of requests required to load a webpage.
6. Optimize Images
Compressing images and using modern formats such as WebP or AVIF can reduce download sizes and improve page-loading speed.
7. Reuse Connections
HTTP persistent connections allow multiple requests to use the same connection, reducing the overhead of repeatedly establishing connections.
HTTP/2 and HTTP/3 also provide mechanisms to improve the efficiency of transferring multiple resources.
8. Use Browser Caching
Browser caching stores eligible resources locally so that they can be reused on subsequent visits, reducing unnecessary downloads.

Performance Optimization Summary

Optimization	**Benefit**
DNS caching	Reduces repeated DNS lookups
CDN	Reduces content delivery latency
Server optimization	Reduces processing time
Data compression	Reduces response size
Image optimization	Speeds up image loading
Browser caching	Avoids unnecessary downloads
Persistent connections	Reduces connection overhead

Efficient JavaScript and CSS	Improves rendering performance

By applying these techniques, developers can reduce loading time, improve responsiveness, and provide a better user experience.

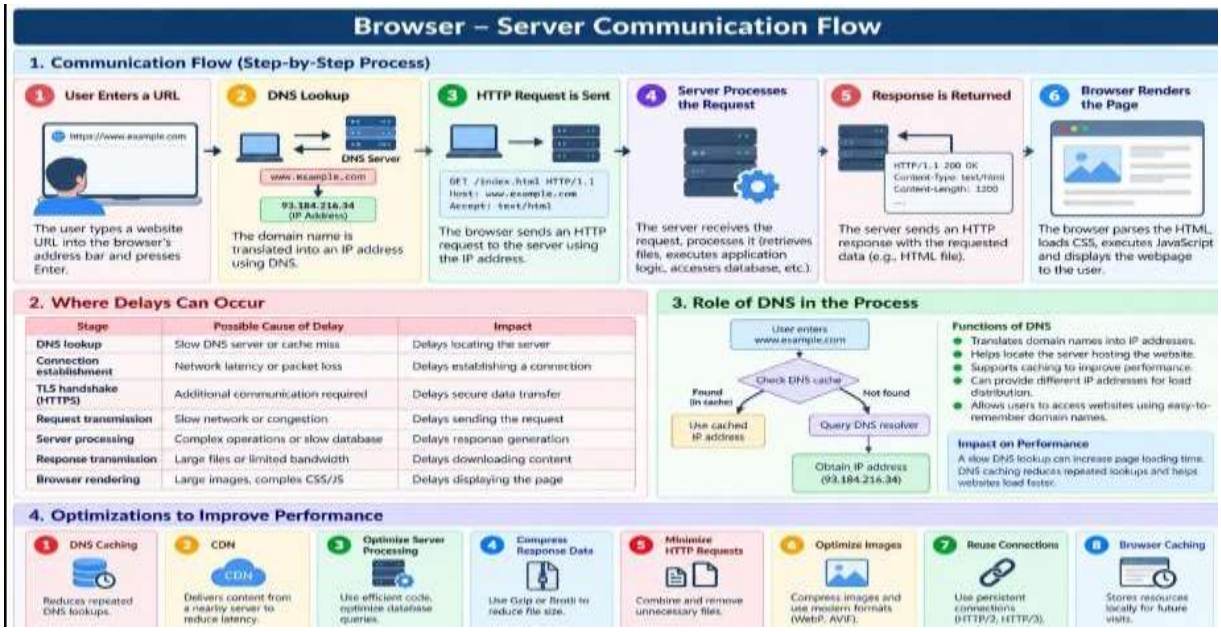


Figure 5: Browser–Server Communication Flow and Performance Optimization

Conclusion

Browser–server communication involves six main stages: entering a URL, performing DNS resolution when necessary, sending an HTTP request, processing the request on the server, receiving the response, and rendering the webpage.

Delays may occur during DNS resolution, connection establishment, server processing, data transmission, or browser rendering. DNS plays an important role by translating domain names into IP addresses and supporting efficient server discovery.

Performance can be improved through DNS and browser caching, CDN usage, server optimization, compression, image optimization, and efficient connection management.

Therefore, understanding browser–server communication and optimizing each stage are essential for developing fast, reliable, and efficient web applications.